EP-L2: Decoupling Tag Residency from Dirty-Data Lifetime in GPU L2 Caches

Author information to be completed
English working draft EN-v7 — not for submission

***Abstract*—Eviction ends tag residency before a dirty line's obligations end: data must survive until downstream acceptance, and address ordering may persist until writeback completion. A fixed-partition L2 bridges these lifetimes by copying unchanged bytes into a writeback buffer. EP-L2 instead transfers ownership: the victim retains its original banked slot, the incoming resident obtains a distinct legal landing, and local data and ordering state retire separately. At equal tag count, payload capacity, and modeled bank service, five complete non-atomic workload pairs eliminate all resident-to-writeback copies but yield only a 1.001600 geometric-mean speed ratio, including one slowdown. Under matched downstream controls, neither design records local-buffer refusals, yet U-WB's relative performance does not uniformly improve. These results distinguish reduced internal work from application progress; competitive static allocation and physical implementation cost remain unresolved.**

***Index Terms*—GPU, L2 cache, writeback, tag–data decoupling, resource sharing**

I. Introduction

Eviction does not end every obligation of a cache line at once. Its tag can become reusable before its dirty bytes leave local storage; even then, same-address ordering may persist until writeback completes. Coupling storage locations to these roles turns a change of ownership into data movement.

In our fixed-partition GPU L2 reference, replacing a dirty line requires a resident-bank read and writeback-buffer (WBUF) write before the later send read. These operations move unchanged bytes. A full WBUF can also reject replacement despite free storage elsewhere. Such refusals expose a resource mismatch, but do not directly measure application delay.

Our insight is to treat eviction as an ownership transfer. The victim keeps its original payload slot while the incoming resident receives another bank-compatible landing. The difficulty is maintaining one transaction across bank refusals and intervening fills: victim identity, landing, and tag commit must agree. After commit, a real lower data carrier must take responsibility for the bytes before local storage is freed; writeback ordering retires later.

EP-L2 realizes this protocol without adding payload capacity or modeled bank ports. Earlier work retains dirty candidates in LLCs, exposes address-only candidates, or separates dirty metadata [1]–[3]. Our narrower question is ownership of the original physical payload after its resident tag is reused.

The evaluation separates internal work from execution. Five matched workloads eliminate copies and three avoid the reference's WBUF-data limit, yet average speed-ratio gain is only 0.160%, with a CFD regression. Matched downstream controls examine when these local savings matter; stronger static partitions and physical cost remain necessary comparisons.

II. Lifetime-Aware L2 Payload Storage

A. Fixed Physical Resources, Different Data Lifetimes

The primary reference, B16 (`B16_BANKED`), partitions 1168 line-sized slots into 1024 resident, 128 bypass-return, and 16 WBUF slots. U-WB16 (`U_WB16`) permits resident and WB allocations across this physical pool. Each slot is 128 B with four 32-B sectors. Both modes provision four banks of 292 slots, at most one modeled line-sized operation per bank per physical GPU cycle, and a common 128-B copy latch outside the pool. Both also retain 128 writeback address-directory (WAD) entries and the same request-metadata limits. Sixteen is the static *data* quota, not the U-WB control limit.

Figure 1 shows the declared service model. WB-only pooling can borrow reference bypass-return slots, but concurrent dynamic bypass use is unvalidated. Idle bypass capacity therefore makes a WB-heavy static control important.

A resident tag names a payload through a slot identifier and generation. Each slot has one authoritative owner; ready/pending sector masks and pins track its consumers. The initial design preserves the resident bank class:

$$\text{bank}(p) = p \bmod 4 = \text{tag_index} \bmod 4. \tag{1}$$

A globally free slot is therefore not necessarily a legal landing for a given tag. The 1024 tags bound resident visibility; a temporarily reserved incoming landing is not an additional visible resident line.

B. Eviction as an Ownership Transfer

For dirty victim A in slot p, B16 reads the resident and writes WBUF before the send read. U-WB16 changes p's owner to `WB_STAGED` and reserves distinct $q \neq p$ for incoming B. The landing must match the tag's bank class. Two copy operations disappear; send read and lower data carriage remain.

The serialized frontend first checks issue-queue/MSHR needs, victim pins and pending sectors, WAD capacity, and legal staging/landing availability. Its pending transaction records requester, victim tag index, old payload handle, and acquired resources. A later probe by the same request uses this held victim, even if another way's fill changes the raw LRU candidate. A different frontend request is refused while this transaction is pending. Bank denial retains its identity and reservations for retry.

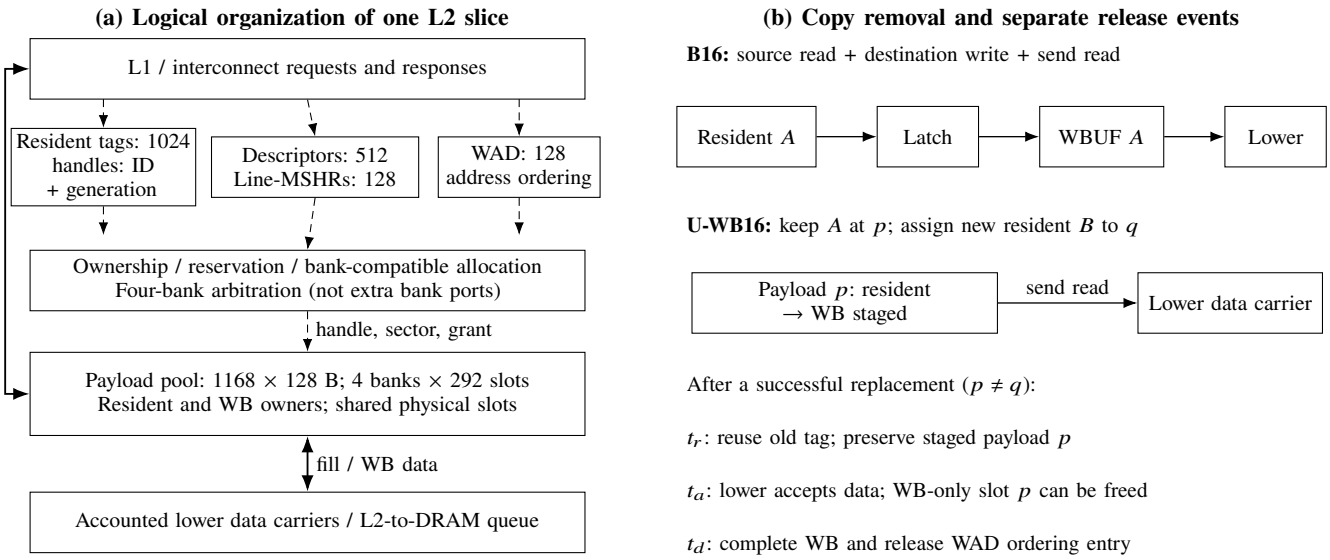


Fig. 1. From relocation to ownership transfer. (a) Resident tags, requester descriptors, and writeback address ordering are separately tracked around a fixed banked payload pool. (b) B16 copies an evicted dirty line to its WBUF; U-WB retains the bytes at p while allocating a different landing q for the incoming resident. Local storage is released after lower data acceptance, whereas address ordering persists to writeback completion. Both modes have equal data capacity, bank service, and a common copy latch. TVD/RO and concurrent dynamic bypass are outside the reported WB-only evaluation.

In B16, a common latch bridges the victim read and staging write. In U-WB16, the reserved incoming landing remains separate from the old dirty slot. Any incoming write obtains its bank grant before ownership is changed. The controller provisionally installs the landing, then invokes underlying tag access; refusal rolls back uncommitted reservations and restores the saved victim/tag identity. Consumed bank operations remain charged. Successful underlying access binds the dirty writeback and clears the pending transaction. Directed tests exercise intervening fills and cross-cycle victim identity.

C. Progress and Independent Retirement

At successful replacement t_r, the old tag becomes reusable while p remains protected (Fig. 1). After a granted send read and actual lower-FIFO push at t_a, the packet carries the dirty data and local p is released. WAD address ordering remains until writeback completion t_d. A late event follows its own handle/transaction identity, not the reused tag.

Requester descriptors and pins survive until response acceptance or internal consumption. Generation identity and per-sector readiness prevent a ready sector from implying a ready line. Eventual lower service and stable bank retry identity are necessary for local progress; whole-GPU deadlock freedom is not established.

D. Implementation Cost and Extension Boundary

The pool holds 146 KiB/slice, or 9.125 MiB across 64 slices; common 128-B latches add 8 KiB/chip. An 11-bit index identifies a slot. Tags need $1024(11+G)$ handle bits for generation width G, and up to 128 WB handles need $128(11+G)$ bits plus control state. These gross representations are not incremental area: the reference already has common fields.

Added control includes bank-compatible allocation, handle/owner checks, pins, and rollback. Concrete SRAM ports, generation bounds, pipelines, and lower data carriers require implementation. Equal modeled capacity/service does not establish equal hit latency, timing, area, or energy; no RTL synthesis, STA, or power measurement is available. TVD and oracle-based RO are disabled, and neither has qualified application results here; Ideal diagnostics are also outside this evaluation.

III. Experimental Methodology

A. Matched Resources and Complete Inputs

We use the Accel-Sim SASS flow and GPGPU-Sim performance-model lineage [4], [5]. Their published validation does not validate this modified L2. B16 and U-WB16 execute with the same independently qualified binary, complete trace input, non-mode settings, timing scope, and termination rules. Table I records the modeled configuration, including the modified L2; it is not a reconstruction of undocumented commercial-cache internals.

The fixed development set is convolutionSeparable, FWT_7_21, cfd_097k, vectorAdd_4M, and SAD. Artifacts bind each complete commandlist, input size, and trace body. SAD is the predeclared negative control. All five pairs, including slowdowns, enter the geometric mean. This development set does not establish holdout or general GPU coverage.

B. Validation Boundary

Independent tests cover pending-victim identity, physical-cycle bank grants, host prefill, symbolic 32-B sector versions, response backpressure, and separate WB acceptance/completion. Every run closes all 64 slices' outstanding requests and resource trackers. Complete opcode auditing restricts application

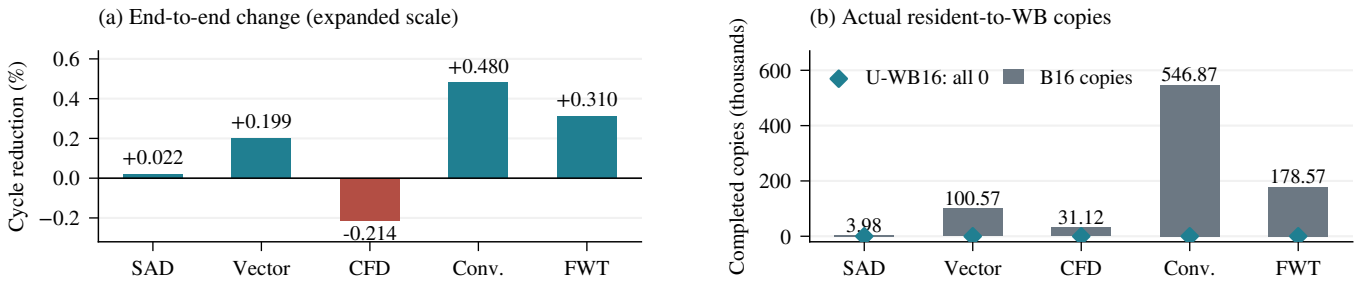


Fig. 2. Five H0 workload pairs; sensitivity results are reported separately. (a) Cycle reduction $100(C_B - C_U)/C_B$, on an expanded scale with zero denoting B16; CFD regression is retained. The speed-ratio geomean is 1.001600, not the mean of these percentages. (b) Actual completed 128-B copies; U-WB16 is zero in every pair. Vector, Conv., and FWT denote vectorAdd_4M, convolutionSeparable, and FWT_7_21. No error bars or significance claim.

TABLE I
DECLARED MATCHED CONFIGURATION. L2 CAPACITIES ARE PER SLICE.

Resource	Setting
Compute / clock domains	80 SMs; core/ICNT/L2: 1132 MHz; DRAM: 850 MHz
L1 (fixed)	64 KiB/SM; 4 banks; 20-cycle setting
L2 tags / data	1024 tags; 1168 slots, 146 KiB
Line / sectors / banks	128 B / 4 × 32 B / 4 × 292 slots
Service / common latch	1 op/bank/physical cycle / 128 B
Descriptors / Line-MSHRs	512 / 128; 32 requesters/address
WAD / lower queue	128 / 128 entries
Chip / memory channels	64 slices / 32 channels
L2 ROP delay	160-cycle model setting
DRAM scheduler / return	128 / 192 entries per channel

qualification to non-atomic inputs. SASS lacks full numerical payloads, so these checks establish transaction/lifecycle consistency rather than numerical output correctness. Reproducibility artifacts bind source, executable, ordered configurations, full commandlists/trace bodies and raw logs; all reported pairs undergo independent input, log, and terminal validation.

IV. From Saved Internal Work to Application Effects

A. Copy Removal Is Not a Speedup Model

All five predeclared H0 workload pairs have complete trace-driven simulation results and pass the stated transaction-lifecycle checks. Each pair has equal instruction counts and matched full input/termination scope. Figure 2(a) reports the complete set: SAD is almost tied; Vector, Convolution, and FWT reduce cycles by 0.199%, 0.480%, and 0.310%, while CFD increases cycles by 0.214%. The geometric mean of C_B/C_U is 1.001600, or a 0.160% speed-ratio gain. These deterministic paired observations establish neither statistical significance nor general GPU speedup.

Figure 2(b) shows removal of every measured resident-to-WB copy. A committed non-rolled-back B16 writeback uses two copy operations and one send read; U-WB16 retains the send read. This local operation saving is not a speedup model. Actual WB counts also change: CFD completes 31,123/31,063 WBs and FWT 178,569/178,585 for B16/U-WB16. Consequently, the run-level logical-operation difference need not equal twice either mode's WB count.

B. Local Staging Relief and Its Downstream Cost

Vector, Convolution, and FWT record 440,161, 845,097, and 890,294 B16 WB-data-full events; U-WB16 records zero. Figure 3(b) shows less modeled bank work and lower aggregate conflict waits. Successful grants are actual model-service observations, not estimates obtained by multiplying copies. At drained termination, their equality to logical operations implies neither elapsed-time nor RTL timing/energy savings.

For the same three workloads, L2-to-DRAM full-block attempts rise from 1.104M to 1.764M, 0.218M to 1.132M, and 0.013M to 0.968M. This is consistent with pressure moving downstream after local staging relief. It does not independently identify the limiting queue or DRAM service. Denials may repeat for one frontend head and overlap other blockers. From final cumulative observations of all 32 channels, the modeled issued-byte/atom-cycle fractions are 64.5/64.8% for Convolution and 14.1/14.1% for FWT. Even frequent queue denials therefore do not prove global DRAM-bandwidth saturation. Kernel-boundary cumulative snapshots are not summed.

CFD is a useful counterexample: despite removing copies and lowering bank waits from 102,546 to 91,029, U-WB16 takes 169 more cycles. WAD-hazard events increase from 9,841 to 13,400, WB counts differ, and successful DRAM writes change from 3,983,744 to 3,976,064 B. These correlated changes motivate ordering/service diagnosis; they do not identify a unique cause. SAD has no observed WB-data-full pressure and differs by only 24 cycles. Neither case is discarded.

C. Separating Buffer Headroom from Service Speed

Q4 expands downstream queues, scheduler/return storage and credits; D4 retains Q4 buffering and raises the DRAM domain (Table II). Both compare B16/U-WB under identical conditions without changing L2 capacity/service. Buffers affect scheduling visibility and cost [6]; D4 shortens physical DRAM timing while preserving the fixed 100-GPU-cycle interface delay. Neither intervention is free hardware or pure bandwidth.

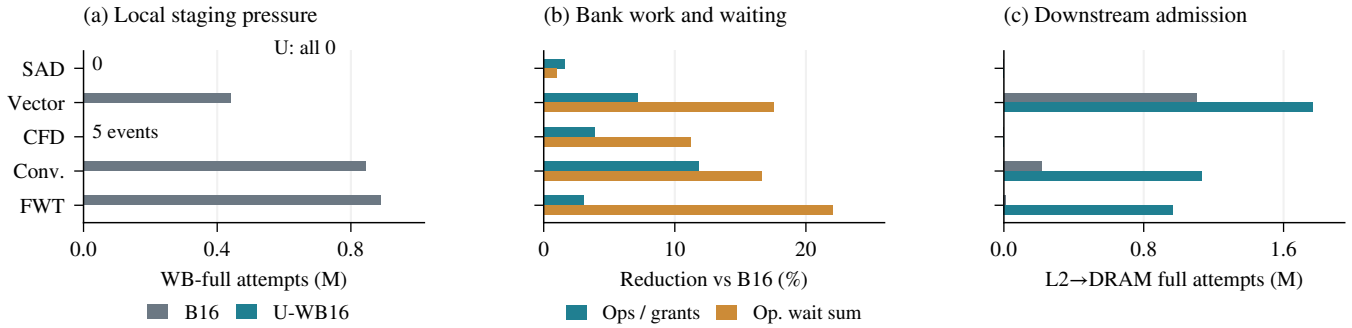


Fig. 3. H0 application-cumulative pressure and service observations. (a) WB-data-full refusal attempts; CFD's small B16 count is labeled explicitly. (b) Separate relative reductions in logical work/successful modeled bank grants and in summed per-operation conflict waits. Logical operations and grants coincide at drain in these runs, but are independently checked counters. (c) Frontend attempts denied when the L2-to-DRAM queue is full. M denotes 10^6. Attempt counts and operation-wait sums are not elapsed GPU cycles or unique transactions; reductions and increases do not establish causality.

TABLE II
MATCHED DOWNSTREAM SENSITIVITY SETTINGS; CORE/ICNT/L2 REMAIN 1132 MHz.

Resource	H0	Q4	D4
L2→DRAM / DRAM→L2	128/64	512/256	512/256
Scheduler / return queues	128/192	512/768	512/768
Derived shared credits	319	1279	1279
DRAM clock (MHz)	850	850	3400

TABLE III
PAIRED CYCLES UNDER MATCHED SERVICE CONDITIONS. $S = C_B/C_U$; $J = S/S(\text{H0})$. Q4 OBSERVATIONS HERE COVER CONVOLUTION ONLY.

Workload, h	C_B	C_U	S	J
Conv., H0	292,932	291,525	1.004826	1.000000
Conv., D4	267,521	268,544	0.996191	0.991406
Conv., Q4	290,333	293,709	0.988506	0.983758
FWT, H0	493,189	491,662	1.003106	1.000000
FWT, D4	452,758	451,254	1.003333	1.000226

For condition h, define $S_w(h) = C_{B,w}(h)/C_{U,w}(h)$ and $J_w(h) = S_w(h)/S_w(\text{H0})$. All four cycles are retained; speeding both modes does not itself improve U-WB's relative effect. Diagnostic means never enter the five-workload H0 mean.

D4 accelerates both modes relative to H0 (Table III), but Convolution reverses U-WB's advantage: it takes 1023 more cycles than B16. FWT saves 1504 cycles, close to its H0 saving of 1527. The two-workload D4 geometric mean is 0.999755×, with interaction 0.995806×; this diagnostic result does not replace the five-workload H0 mean. More downstream headroom does not uniformly amplify the mechanism's benefit.

D4 records no WB-data-full, L2-to-DRAM-full, or scheduler-full attempts in either mode, yet modeled bank-wait sums rise relative to H0. Observed zeros do not establish globally nonbinding service. Q4 Convolution also removes WBUF/lower-queue-full attempts, but U-WB is 3376 cycles slower; WAD-full attempts are 55,645/54,516 for B16/U-WB. Local data can leave well before ordering obligations retire. These repeated event counts identify candidate constraints, not unique stalls or a proven critical path. Staging durations start at B16's copy readout versus U-WB's producer commit, so only same-mode, across-condition comparisons share endpoints. Further attribution requires matched service and ordering evidence.

An equal-total strong static repartition remains unevaluated. Thus, current results combine copy removal and pooling; they do not isolate dynamic elasticity from a better fixed WB quota. Static allocation, service regime, and implementation cost bound the present performance claim.

V. Relation to Prior Storage Decoupling

VWQ exposes dirty LLC candidates to scheduling while retaining a physical write queue [1]. Decoupled last-write prediction tracks candidate addresses separately while data remain in LLC; evicted dirty blocks use another write buffer [2]. DBI groups dirty metadata outside tags by DRAM row [3]. EP-L2 instead tracks the original slot after tag reuse, its distinct incoming landing, and separate payload/order retirement. DBI dirtiness is not WAD's in-flight address ordering.

V-Way separates tag/data placement [7]; Tag-Split Cache changes storage granularity [8]. Unified GPU memory shares SM-local register/cache/scratchpad storage [9]. In-cache MSHRs [10], MiCache's shared FPGA data/MSHR storage [11], and DL-MSHR's linked slots that also hold write-miss data [12] concern different shared resources. Our comparison fixes resident-tag and miss-tracking budgets.

VI. Conclusion

Eviction need not relocate dirty data when a tag becomes reusable. EP-L2 preserves the victim's slot, allocates a legal incoming landing, and retires payload storage separately from writeback ordering. Five complete workloads eliminate copies but yield only a 1.001600 geometric-mean speed ratio. Reduced bank work and staging pressure therefore do not guarantee faster execution. Matched downstream controls do not uniformly amplify U-WB's relative benefit. Competitive static allocation and physical cost remain unresolved.

References

[1] J. Stuecheli, D. Kaseridis, D. Daly, H. C. Hunter, and L. K. John, "The virtual write queue: Coordinating DRAM and last-level cache policies," in *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*, 2010, pp. 72–82.

[2] Z. Wang, S. M. Khan, and D. A. Jiménez, "Improving writeback efficiency with decoupled last-write prediction," in *Proceedings of the 39th Annual International Symposium on Computer Architecture (ISCA)*, 2012, pp. 309–320.

[3] V. Seshadri, A. Bhowmick, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "The dirty-block index," in *Proceedings of the 41st Annual International Symposium on Computer Architecture (ISCA)*, 2014, pp. 157–168.

[4] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-Sim: An extensible simulation framework for validated GPU modeling," in *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 473–486.

[5] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2009, pp. 163–174.

[6] R. Ausavarungnirun, K. K.-W. Chang, L. Subramanian, G. H. Loh, and O. Mutlu, "Staged memory scheduling: Achieving high performance and scalability in heterogeneous systems," in *Proceedings of the 39th Annual International Symposium on Computer Architecture (ISCA)*, 2012, pp. 416–427.

[7] M. K. Qureshi, D. Thompson, and Y. N. Patt, "The V-Way cache: Demand-based associativity via global replacement," in *Proceedings of the 32nd Annual International Symposium on Computer Architecture (ISCA)*, 2005, pp. 544–555.

[8] L. Li, A. B. Hayes, S. L. Song, and E. Z. Zhang, "Tag-split cache for efficient GPGPU cache utilization," in *Proceedings of the 2016 International Conference on Supercomputing (ICS)*, 2016, pp. 43:1–43:12.

[9] M. Gebhart, S. W. Keckler, B. Khailany, R. Krashinsky, and W. J. Dally, "Unifying primary cache, scratch, and register file memories in a throughput processor," in *Proceedings of the 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2012, pp. 96–106.

[10] K. I. Farkas and N. P. Jouppi, "Complexity/performance tradeoffs with non-blocking loads," Digital Equipment Corporation, Western Research Laboratory, Research Report 94/3, Mar. 1994.

[11] S. Xu, S. Lu, Z. Shao, X. Liao, and H. Jin, "MiCache: An MSHR-inclusive non-blocking cache design for FPGAs," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2024, pp. 22–32.

[12] Y. Gu and L. Chen, "Dynamically linked MSHRs for adaptive miss handling in GPUs," in *Proceedings of the International Conference on Supercomputing (ICS)*, 2019, pp. 510–521.